

Report CS163 - Solo Project - Task 2

- **Course:** CS163 - Data Structure
- **Class:** 23TT2
- **Full Name:** Vo Khac Trieu
- **Student ID:** 23125020
- **Tutor:** (PhD) Dinh Ba Tien, (MSc) Ho Tuan Thanh, (MSc) Truong Phuoc Loc

Introduction

Algorithm

Dijkstra

A* / A-star

Strategy

Path Caching

Hierarchical Pathfinding

Landmark-based Pathfinding

Dijkstra

Theoretical Overview

Code Explanation

Conclusion

A*

Theoretical Overview

Basics of A*

Heuristic Function

A* with Path Caching

Introduction to Path Caching

Design and Implementation of A* with Path Caching

Graph Representation

Heuristic Function

Path Caching Mechanism

A* Search Process

Code Implementation

Performance Considerations

Conclusion
Introduction
Hierarchical PathFinding (HPA*)
Theoretical Overview
Concept
Algorithm Design
Implementation Details
Graph Preparation
Hierarchical Pathfinding Class
Conclusion
Comparison of time/space complexity and real-time runtime performance

Introduction

There are many algorithms of find shortest path of two nodes. However, each algorithm have some benefits and drawbacks that noticeable. Below, I will introduce some algorithms and strategies can be use for finding shortest path, thought, effective with the real-world graphs.

Algorithm

Dijkstra

Dijkstra's algorithm is a classical approach for finding the shortest path between nodes in a graph, particularly useful when all edge weights are non-negative. It begins at the source node, exploring all neighboring nodes, and iteratively selects the node with the smallest tentative distance, updating the shortest path estimates until the destination node is reached or all nodes have been processed.

- **Optimality:** Guarantees the shortest path in graphs with non-negative weights.
- **Complexity:** Typically $O(V^2)$ using an adjacency matrix or $O((V + E) \log V)$ with a priority queue, where V is the number of vertices and E is the number of edges.

A* / A-star

A* (A-star) is an extension of Dijkstra's algorithm that incorporates a heuristic to guide the search towards the target node more efficiently. It balances the path

cost from the start node with an estimated cost (heuristic) to reach the goal, reducing the number of nodes explored.

- **Heuristic Function:** Uses a function $h(n)$ that estimates the shortest path from a node n to the goal, typically the straight-line distance in spatial graphs.
- **Optimality:** Guarantees the shortest path if the heuristic is admissible (never overestimates) and consistent (follows the triangle inequality).
- **Complexity:** Similar to Dijkstra's but can be faster depending on the quality of the heuristic, often $O((V + E) \log V)$.

Strategy

Although these algorithms have a good complexity with one query of shortest path finding, if we have lots of queries, we should try to optimize by strategies based on pre-calculation or use the result of before queries.

Path Caching

Path caching is a technique used to optimize pathfinding by storing and reusing previously computed paths. This method is especially useful in scenarios where certain paths are frequently requested, allowing the system to bypass redundant computations.

Hierarchical Pathfinding

Hierarchical Pathfinding A* (HPA*) is an optimized version of the A* algorithm, designed to handle large-scale graphs by dividing them into smaller, manageable regions (or clusters). The core idea is to abstract the graph into multiple levels of hierarchy, where each level simplifies the graph by treating clusters of nodes as single units.

Landmark-based Pathfinding

Landmark-based pathfinding is an optimization technique used in large-scale navigation systems like Google Maps. The algorithm relies on preselected landmarks, which are strategically placed nodes in the graph. The shortest paths between these landmarks are precomputed and cached, allowing for faster pathfinding by leveraging these cached paths.

Although this strategy is good, I still stuck to implement this algorithm, so right now my report do not have this strategy.

Dijkstra

Theoretical Overview

Dijkstra's algorithm is a greedy algorithm that solves the single-source shortest path problem for a graph with non-negative edge weights. The algorithm works by iteratively selecting the node with the smallest known distance from the source and updating the distances of its neighbors. This process continues until the shortest path to the destination node is found.

Code Explanation

My implementation of Dijkstra's algorithm is structured in the following key components:

Graph Construction (`get_graph()`):

- The function `get_graph()` reads the graph structure from a file (`Graph.txt`) and constructs an adjacency list representing the graph.
- Each vertex's adjacency list contains tuples representing the neighboring vertices and the edge lengths (weights) connecting them.
- The graph is built using the `graph_building` module, which encapsulates the graph's vertices and edges.

```
def get_graph():
    G = graph_building.graph_building_from_file("Graph.txt")
    adj_list = {}
    for v in G._vertices_list:
        adj_list[v._id] = []
    for e in G._edges_list:
        if (e._start._id == e._stop._id):
            continue
        adj_list[e._start._id].append((e._stop._id, e._leng
```

```
th))  
    return G, adj_list
```

Initialization:

- In the `Dijkstra_Shortest_Path()` function, the algorithm begins by initializing two dictionaries: `dist` for storing the minimum distances from the source vertex `u` to all other vertices and `par` for tracking the parent of each vertex in the shortest path tree.
- Initially, all distances are set to infinity (`math.inf`), except for the source vertex `u`, which is set to 0.

```
dist = {}  
par = {}  
for v in G._vertices_list:  
    dist[v._id] = math.inf  
    par[v._id] = v._id  
dist[U] = 0
```

Priority Queue:

- A priority queue (`pqueue`) is used to efficiently retrieve the vertex with the smallest distance during each iteration. This is implemented using Python's `heapq` module.
- The source vertex `u` is added to the priority queue with a distance of 0.

```
pqueue = []  
heapq.heappush(pqueue, (0, U))
```

Main Loop:

- The main loop of the algorithm continuously processes vertices from the priority queue until it is empty. For each vertex `u`, the algorithm checks its adjacent vertices `v`.
- If a shorter path to `v` is found through `u`, the distance is updated, and `v` is added to the priority queue with its updated distance.

```

while pqueue:
    du, u = pqueue[0]
    heapq.heappop(pqueue)

    if abs(du - dist[u]) > 1e-4:
        continue

    for v, length in adj_list[u]:
        if dist[u] + length < dist[v]:
            dist[v] = dist[u] + length
            par[v] = u
            heapq.heappush(pqueue, (dist[v], v))

```

Path Reconstruction:

- After the algorithm completes, the shortest path from `u` to `v` is reconstructed by backtracking from `v` using the `par` dictionary.
- If the distance to `v` is still infinity, it indicates that no path exists between `u` and `v`.

```

if dist[V] == math.inf:
    return math.inf, []

t = V
path = [V]
while t != U:
    t = par[t]
    path.append(t)
path.reverse()

return dist[V], path

```

Conclusion

Although, this is one of the best algorithm for finding shortest path problem, but when deal with multiple queries, this algorithm can not use the old results to calculate.

A*

Theoretical Overview

Basics of A*

The A* algorithm is a widely used pathfinding and graph traversal algorithm. It is an extension of Dijkstra's algorithm, adding a heuristic to guide the search more efficiently towards the goal. The A* algorithm finds the shortest path between a start and a goal node by considering both the cost to reach a node (g value) and an estimated cost to reach the goal from that node (h value). The total cost (f value) is calculated as:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ is the cost of the path from the start node to node n .
- $h(n)$ is the heuristic that estimates the cost from node n to the goal.

Heuristic Function

The heuristic function is crucial to the efficiency of A*. It provides an estimate of the distance between the current node and the goal. A common heuristic used in geographical applications is the Manhattan distance, calculated as:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

This heuristic is particularly effective in grid-based maps or when nodes represent geographical coordinates, as it provides a reasonable approximation of the actual distance.

A* with Path Caching

Introduction to Path Caching

Path caching is an optimization technique used to enhance the performance of pathfinding algorithms by storing paths that have been computed previously. When a new path request is made, the algorithm first checks if the path is already in the cache. If it is, the cached path is returned immediately, avoiding the need to recompute the path.

Design and Implementation of A* with Path Caching

The A* with Path Caching algorithm is implemented as follows:

Graph Representation

The graph is represented as an adjacency list, where each node points to a list of its neighboring nodes and the corresponding edge lengths. Additionally, a mapping of node IDs to their geographical coordinates (latitude and longitude) is maintained.

Heuristic Function

The heuristic used in this implementation is the Manhattan distance, calculated based on the geographical coordinates of the nodes. This heuristic guides the search by providing an estimate of the remaining distance to the goal.

Path Caching Mechanism

- **Cache Structure:** The cache is implemented as a dictionary, where the keys are tuples representing the start and goal nodes, and the values are the computed paths.
- **Cache Lookup:** Before performing the A* search, the algorithm checks if the path between the given start and goal nodes is already in the cache. If found, the cached path is returned, significantly reducing the computation time.
- **Cache Update:** If the path is not in the cache, the A* algorithm is executed, and the resulting path is stored in the cache for future use.

A* Search Process

The A* search is performed as follows:

1. **Initialization:** The open list (priority queue) is initialized with the start node, and the `g_score` and `f_score` dictionaries are set up.
2. **Node Expansion:** Nodes are expanded based on the lowest `f_score`. For each neighbor, the tentative `g_score` is calculated, and if it is better than the existing score, the neighbor is added to the open list.
3. **Path Reconstruction:** Once the goal node is reached, the path is reconstructed by tracing back through the `came_from` dictionary.
4. **Caching:** The computed path is stored in the cache for future queries.

Code Implementation

Here is the Python implementation of the A* with Path Caching algorithm:

```
import graph_building
from graph_building import Graph
import math
import heapq
import time, random

mp = {}

def get_graph():
    G = graph_building.graph_building_from_file("Graph.txt")
    adj_list = {}
    for v in G._vertices_list:
        adj_list[v._id] = []
        mp[v._id] = v
    for e in G._edges_list:
        if e._start._id == e._stop._id:
            continue
        adj_list[e._start._id].append((e._stop._id, e._length))
    return G, adj_list

class AStarPathCaching:
```

```

def __init__(self, graph):
    self.graph = graph # Graph represented as an adjacency list
    self.cache = {} # Cache to store computed paths

def heuristic(self, start, goal):
    return abs(mp[start]._lat - mp[goal]._lat) + abs(mp[start]._lng - mp[goal]._lng)

def a_star_search(self, start, goal):
    if (start, goal) in self.cache:
        return self.cache[(start, goal)] # Return cached path if it exists

    open_list = []
    heapq.heappush(open_list, (0, start))
    came_from = {}
    g_score = {start: 0}
    f_score = {start: self.heuristic(start, goal)}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = self.reconstruct_path(came_from, current)
            self.cache[(start, goal)] = path # Cache the computed path
            return g_score[goal], path

        for neighbor, length in self.graph[current]:
            tentative_g_score = g_score[current] + length
            if neighbor not in g_score or tentative_g_score < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g_score

```

```

        f_score[neighbor] = g_score[neighbor] + s
        elf.heuristic(neighbor, goal)
        heapq.heappush(open_list, (f_score[neighbor], neighbor))

    return -1, [] # Return -1 if no path is found

def reconstruct_path(self, came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    return path[::-1] # Return reversed path

G, adj_list = get_graph()
astar = AStarPathCaching(adj_list)

t_start = time.time()
for i in range(1000):
    print(i)
    dist, path = astar.a_star_search(random.randint(0, 3000
0), random.randint(0, 30000))
    t_end = time.time()
    print(t_end - t_start)

```

Performance Considerations

The performance of the A* with Path Caching algorithm depends heavily on the graph's size, the frequency of repeated path requests, and the efficiency of the cache lookup and storage. In scenarios where the same paths are frequently requested, this approach can significantly reduce the time complexity from $O(N)$ to $O(1)$ for cached paths.

However, the memory consumption increases as more paths are cached, requiring careful cache management to avoid excessive memory usage. Techniques such

as Least Recently Used (LRU) caching can be applied to maintain an efficient cache size.

Conclusion

The A* with Path Caching algorithm represents an effective enhancement to the traditional A* algorithm, particularly in environments where pathfinding queries are frequent and repetitive. By leveraging path caching, the algorithm reduces the need for redundant computations, leading to faster and more efficient pathfinding.

Introduction

Hierarchical Pathfinding A* (HPA*) is an advanced pathfinding algorithm designed to optimize the search process in large-scale graphs by dividing the graph into smaller regions. This approach leverages a hierarchical structure to simplify pathfinding, making it more efficient for complex and extensive networks. This report presents a detailed overview of the HPA* implementation, including its design, implementation, and performance analysis.

Hierarchical PathFinding (HPA*)

Theoretical Overview

Concept

Hierarchical Pathfinding (HPA*) improves the efficiency of pathfinding by breaking down a large graph into smaller, more manageable regions. By using this hierarchical approach, the algorithm can quickly navigate through the graph, focusing on higher-level regions before delving into the detailed pathfinding within smaller regions.

Algorithm Design

Graph Representation:

- The graph is represented as an adjacency list where each node points to its neighboring nodes and associated edge lengths.

- Nodes are divided into regions based on a specified `region_size`, creating a hierarchical structure.

Path Caching:

- Paths between border nodes of different regions are precomputed and cached. This reduces the need for recalculating paths between regions and speeds up the overall pathfinding process.

Implementation Details

Graph Preparation

The `get_graph()` function reads the graph data from a file and constructs an adjacency list for the graph. It also maps each node ID to its corresponding vertex object.

```
def get_graph():
    G = graph_building.graph_building_from_file("Graph.txt")
    adj_list = {}
    for v in G._vertices_list:
        adj_list[v._id] = []
        mp[v._id] = v
    for e in G._edges_list:
        if (e._start._id == e._stop._id):
            continue
        adj_list[e._start._id].append((e._stop._id, e._length))
    return G, adj_list
```

Hierarchical Pathfinding Class

Initialization:

- **Regions:** Nodes are divided into regions based on the `region_size`. This is done by grouping nodes into regions, where each region contains nodes within a certain range.

- **Borders:** Border nodes are identified as nodes that connect different regions. These nodes are crucial for connecting paths between regions.
- **Cache:** Paths between border nodes of different regions are computed and stored in a cache for quick retrieval.

```
class HierarchicalPathfinding:
    def __init__(self, graph, region_size):
        self.graph = graph # Graph represented as an adjacency list
        self.region_size = region_size # Size of each region
        self.regions = self.divide_into_regions() # Divide the graph into regions
        self.borders = self.identify_borders() # Identify border nodes in each region
        self.border_paths_cache = self.cache_border_paths()
        # Cache paths between borders
```

Region Division:

Nodes are divided into regions based on their ID, with each region containing nodes within a specified size.

```
def divide_into_regions(self):
    regions = {}
    region_id = 0
    for node in self.graph:
        region_id = node // self.region_size
        if region_id not in regions:
            regions[region_id] = []
        regions[region_id].append(node)
    return regions
```

Border Identification:

Border nodes are nodes that have neighbors in different regions. These nodes are essential for connecting paths between regions.

```

def identify_borders(self):
    borders = {}
    mp = {}
    for region_id, nodes in self.regions.items():
        borders[region_id] = []
        for node in nodes:
            for neighbor, len in self.graph[node]:
                neighbor_region = neighbor // self.region_size

                if (mp.get((neighbor_region, region_id)) == None):
                    mp[(neighbor_region, region_id)] = 1
                elif (mp[(neighbor_region, region_id)] <= 9):
                    mp[(neighbor_region, region_id)] += 1
                else:
                    continue
            if neighbor_region != region_id:
                borders[region_id].append(node)
    return borders

```

Caching Border Paths:

Paths between border nodes of different regions are precomputed and cached. This reduces computation time during pathfinding queries.

```

def cache_border_paths(self):
    cache = {}
    l = len(self.borders)
    for i in range(l):
        for j in range(i + 1, l):
            border_nodes_l = self.borders[i]
            border_nodes_r = self.borders[j]
            for border1 in border_nodes_l:
                for border2 in border_nodes_r:
                    if border1 != border2:
                        Len, path = self.a_star_search(border

```

```

1, border2)
                                cache[(border1, border2)] = (Len, pat
h)
    return cache

```

*A Search Implementation:**

The A* search algorithm is used to find the shortest path between nodes. It considers both the actual cost (**g**) and the heuristic estimate (**h**).

```

def a_star_search(self, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))
    came_from = {}
    g_score = {start: 0}
    f_score = {start: self.heuristic(start, goal)}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            return g_score[goal], self.reconstruct_path(came_
from, current)

        for neighbor, len in self.graph[current]:
            tentative_g_score = g_score[current] + len
            if neighbor not in g_score or tentative_g_score <
g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g_score
                f_score[neighbor] = g_score[neighbor] + self.
heuristic(neighbor, goal)
                heapq.heappush(open_list, (f_score[neighbor],
neighbor))

    return math.inf, []

```

Path Reconstruction:

The path is reconstructed from the `came_from` dictionary once the goal node is reached.

```
def reconstruct_path(self, came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    return path[::-1]
```

Finding Paths Between Regions:

When start and goal nodes are in different regions, the algorithm finds paths through border nodes and combines them to form the complete path.

```
def find_path(self, start, goal):
    start_region = start // self.region_size
    goal_region = goal // self.region_size

    if start_region == goal_region:
        return self.a_star_search(start, goal) # Same region, use A* directly

    # Different regions, find path through borders
    shortest_path = None
    len_shortest_path = math.inf

    for start_border in self.borders[start_region]:
        for goal_border in self.borders[goal_region]:
            len1, start_to_border = self.a_star_search(start, start_border)
            len2, border_to_border = self.border_paths_cache.get((start_border, goal_border))
            len3, goal_border_to_goal = self.a_star_search(goal, goal_border)
```

```

        if len1 + len2 + len3 == math.inf:
            return math.inf, []
        full_path = start_to_border + border_to_border
        [1:] + goal_border_to_goal[1:]

        if shortest_path is None or len1 + len2 + len3 <
len_shortest_path:
            shortest_path = full_path
            len_shortest_path = len1 + len2 + len3

    return len_shortest_path, shortest_path

```

Conclusion

The Hierarchical Pathfinding (HPA*) algorithm presented here provides an efficient solution for pathfinding in large-scale graphs. By dividing the graph into regions, identifying border nodes, and caching paths between borders, the algorithm significantly reduces computation time for finding paths.

The implementation demonstrates the effectiveness of hierarchical pathfinding in managing extensive graphs and optimizing search operations. The use of caching further enhances performance by reducing redundant computations. This approach is well-suited for applications requiring frequent and efficient pathfinding queries.

Comparison of time/space complexity and real-time runtime performance

Algorithm	Time complexity	Space complexity	Real-time runtime (1000 queries)
Dijkstra	$O(E \log V)$	$O(E + V)$	49 seconds
A* with path caching	$O(E \log V)$, but $O(1)$ if the results in the	$O(E + V + M)$, with M is the size of the	19 seconds

	cache	cache	
Hierarchical Pathfinding	Assume b is region size: - Precalculate: $O((\frac{b}{V})^2 \times E \log V)$ - Query: Smaller than $O(b^2 + b \log V)$	$O(E + V + b^2 + (\frac{b}{V})^2)$	$b = 1000$: 12 seconds